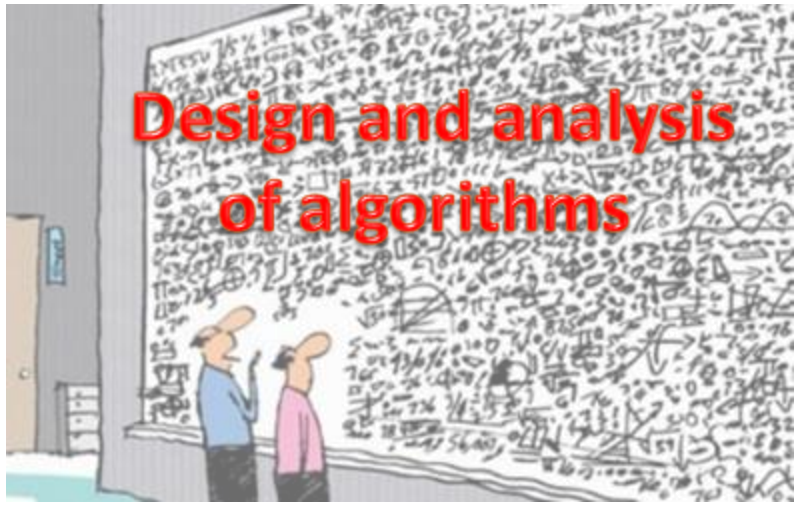
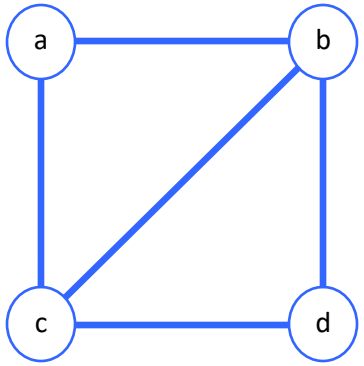


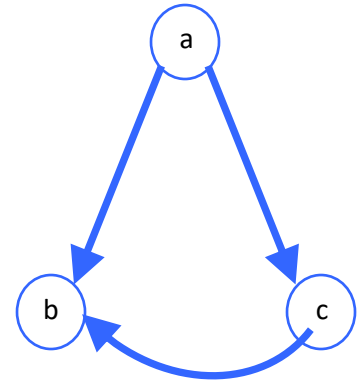
- **Graph representations**

Objective

- What are graphs and how we represent them.
- Motivate graph searches
- For the material of this week:
  - Read DPV Chapter 3 (up to 3.4) and Chapter 4.1-4.2.
  - Same material in CLRS 22.1-22.2.

Graphs

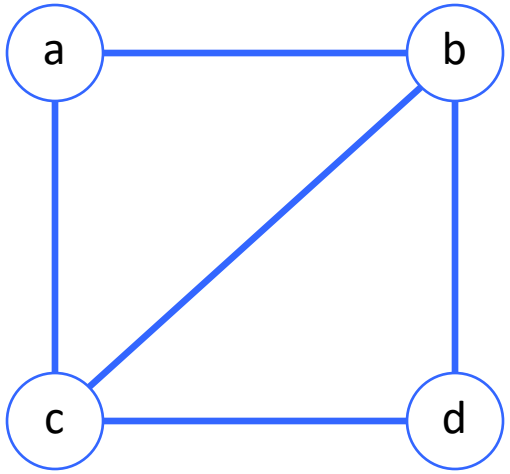




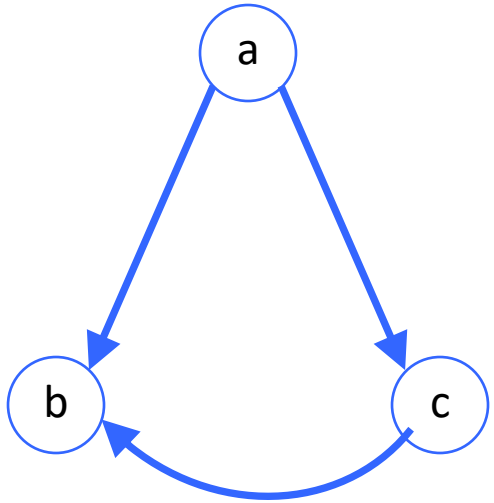
- $G = (V, E)$.
- V : a set of vertices.
  - $|V|$ usually denoted by n.
- $E \subseteq V \times V$: a set of edges (pairs of vertices).
  - $|E|$ usually denoted by m.
  - Note: $m \leq n^2 = O(n^2)$.
- Two flavors:
  - Pay attention to order of vertices in edge: *directed* graph.
  - Ignore order: *undirected* graph.
    - Then only $n(n-1)/2$ possible edges.

Examples

- Undirected
- $V = \{a, b, c, d\}$
- $E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}, \{c, d\}\}$



- Directed
- $V = \{a, b, c\}$
- $E = \{(a, c), (a, b), (c, b)\}$



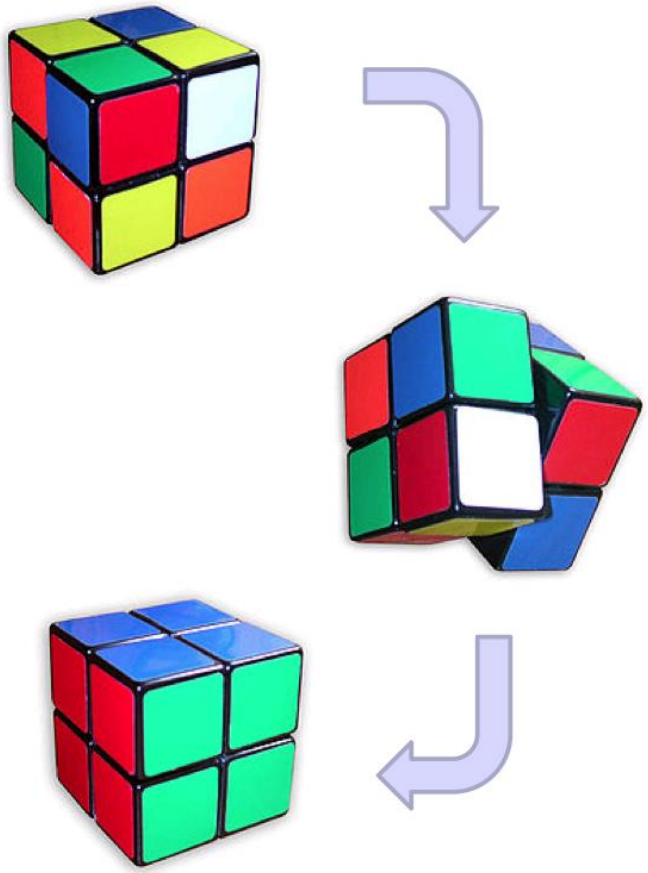
Instances/applications

- Web
  - crawling
- Social Network
  - friend finder
- Computer Networks
  - internet routing
  - connectivity
- Game states
  - Rubik’s cube, chess

Graphs can model state transitions

example by C. Daskalakis, MIT slides

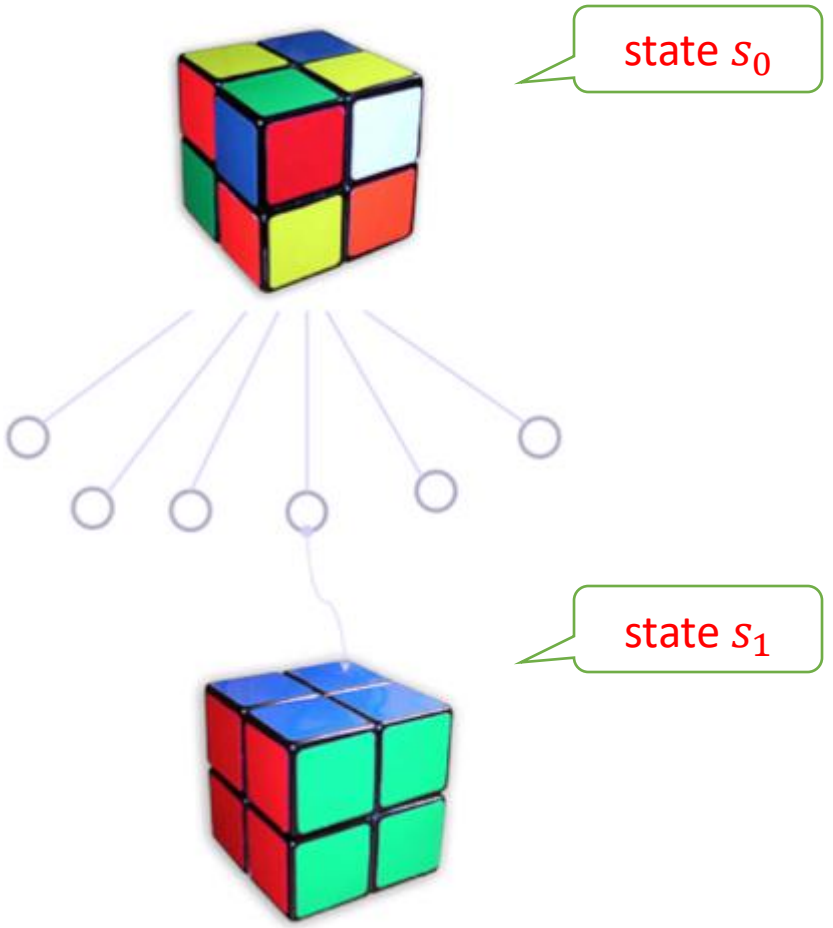
Pocket Cube



- $2 \times 2 \times 2$ Rubik's cube
- Start with a given configuration
- Moves are quarter turns of any face
- "Solve" by making each side one color

State (configuration) graph

- Imagine a graph that has:
  - **One vertex** for each **state** (configuration) of the cube.
  - **One edge** for each **move from a vertex**
    - 6 faces to twist.
    - 3 non-trivial ways to twist (1/4, 2/4, 3/4).
    - Hence 18 edges out of each state.
- Solve the puzzle by finding a path (sequence of moves) from **initial** state to "**solved**" state.



Combinatorics

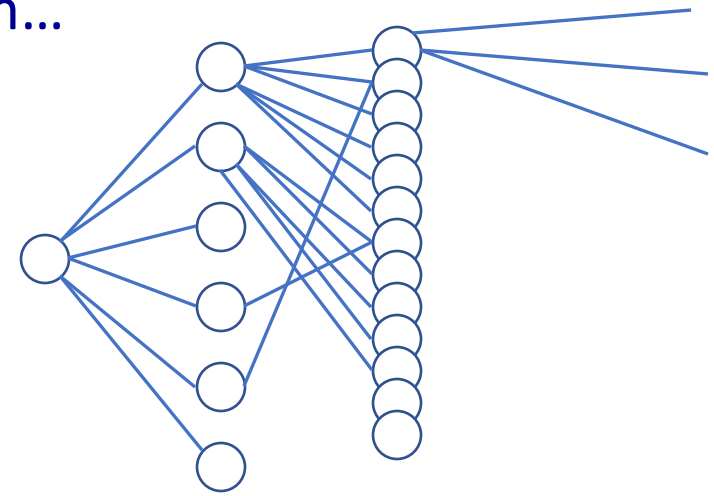
- Different state for each arrangement and orientation of 8 cubelets.
  - 8 cubelets in each position: 8! Possibilities.
  - Each cube has 3 orientations: 3^8 Possibilities.
  - Total: $8!\ 3^8 = 264{,}539{,}320$ vertices.
- But divide out 24 orientations of whole cube.
- And there are three separate connected components (twist one cube out of place 3 ways).
- Result: 3,674,160 states to search.



State exploration

- One start vertex
- 6 others reachable by
  one 90° turn
- From those, 27 others by
  another 90° turn…
- And so on…

New states



distance	90°	90° and 180°
0	1	1
1	6	9
2	27	54
3	120	321
4	534	1847
5	2,256	9,992
6	8,969	50,136
7	33,058	227,526
8	114,149	870,072
9	360,508	1,887,748
10	930,588	623,800
11	1,350,852	2,644
12	782,536	
13	90,280	
14	276	

diameter

Another example of the graph being implicit

Consider the transition function

$$x' = \begin{cases} (x+3) \bmod 11 & \text{if } x = 2k, k \geq 0 \\ (x+2) \bmod 11 & \text{if } x = 2k+1, k \geq 0 \\ x - 1 & \text{if } x > 1 \end{cases}$$

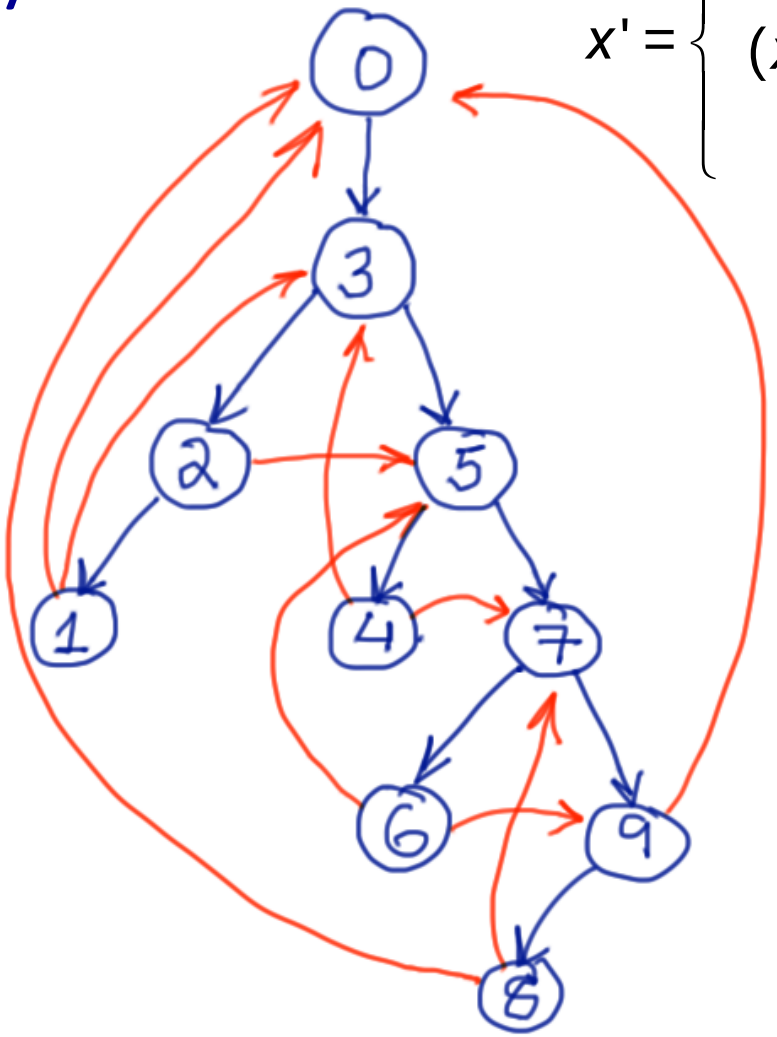
the transition function maps states to sets of states

A. Is 10 reachable from 0?
B. Which is the minimum number of steps needed to reach 4?

Solution: construct the reachability graph!

Reachability tree

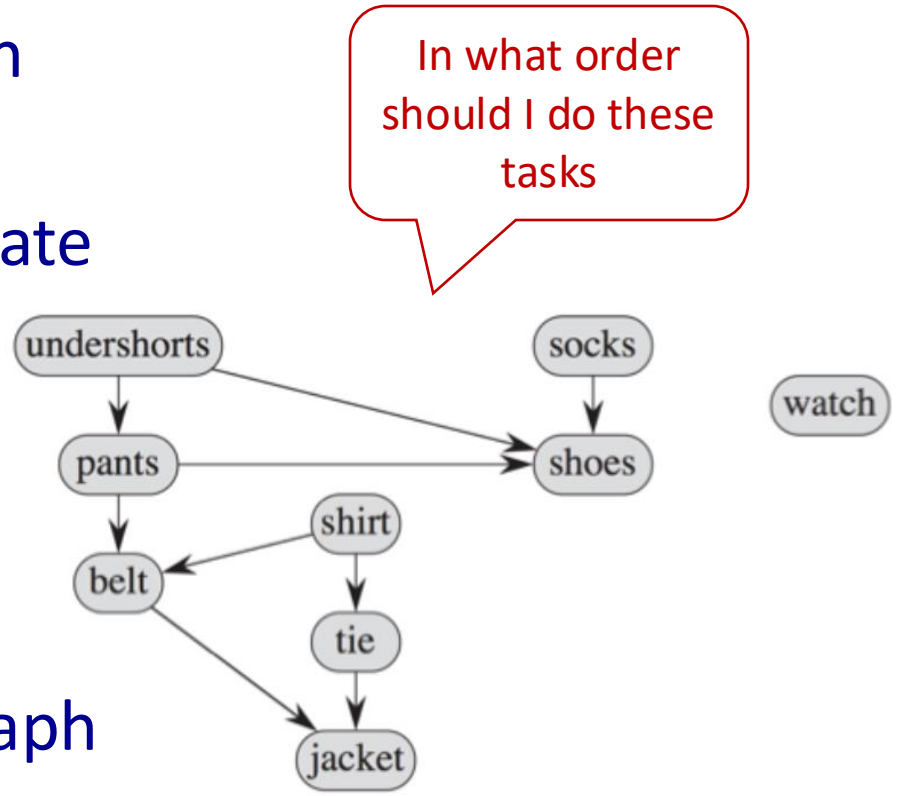
$$x' = \begin{cases} (x+3) \bmod 11 & \text{if } x = 2k, k \geq 0 \\ (x+2) \bmod 11 & \text{if } x = 2k+1, k \geq 0 \\ x - 1 & \text{if } x > 1 \end{cases}$$



shortest-path tree

Problems that relate to graphs

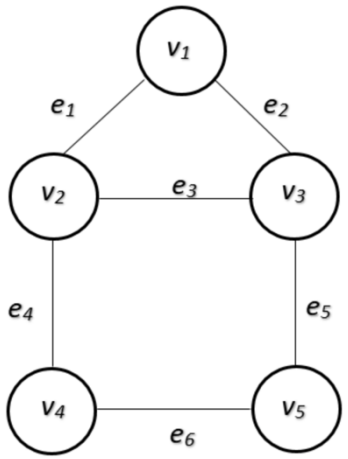
- Reachability: find all states reachable from a given state.
- Shortest paths: find the shortest path between state s_o and s_d.
- Find if a directed graph has a cycle.
- Topological sort: find a "causal ordering" of the nodes *if it exists*.
- Shortest spanning tree: find the "shortest" subgraph that contains all nodes.
- Strongly connected components: maximal sets of nodes that from every node can reach any other node in the set (mixing).
- Many other….



Graph representation

- To solve graph problems, must examine graph.
- Need to represent in computer.
- Four representations with pros/cons.
  - Adjacency lists (of neighbors for **each vertex**).
  - Incidence lists (of edges from **each vertex**).

  } similar
  - Adjacency matrix (0-1, which pairs are adjacent).
  - Implicit representation (as a "find neighbor" function).



v_1	v_2, v_3
v_2	v_1, v_3, v_4
v_3	v_1, v_2, v_5
v_4	v_2, v_5
v_5	v_3, v_4

adjacency list

	v_1	v_2	v_3	v_4	v_5
v_1	0	1	1	0	0
v_2	1	0	1	1	0
v_3	1	1	0	0	1
v_4	0	1	0	0	1
v_5	0	0	1	1	0

adjacency matrix

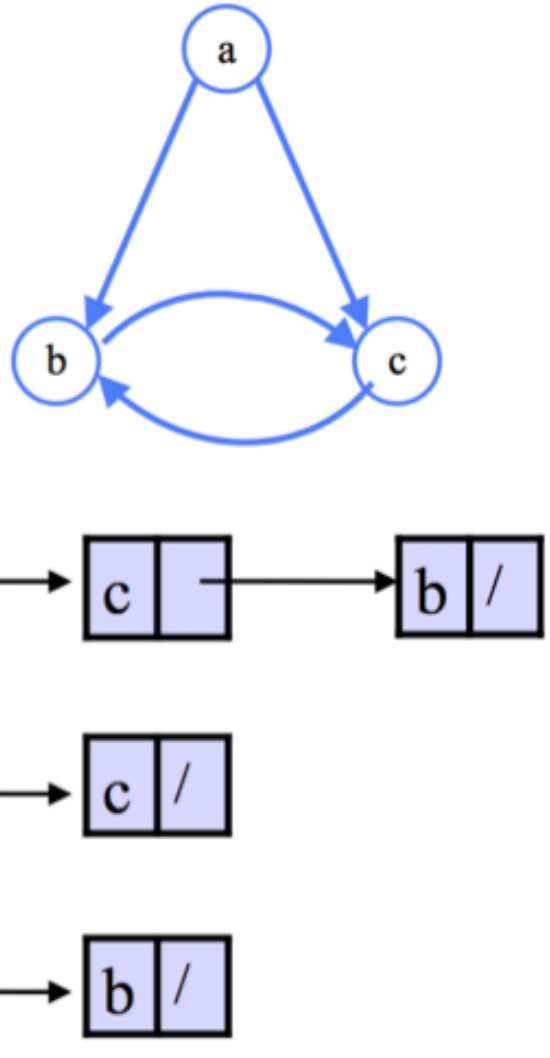
	e_1	e_2	e_3	e_4	e_5	e_6
v_1	1	1	0	0	0	0
v_2	1	0	1	1	0	0
v_3	0	1	1	0	1	0
v_4	0	0	0	1	0	1
v_5	0	0	0	0	1	1

incidence matrix

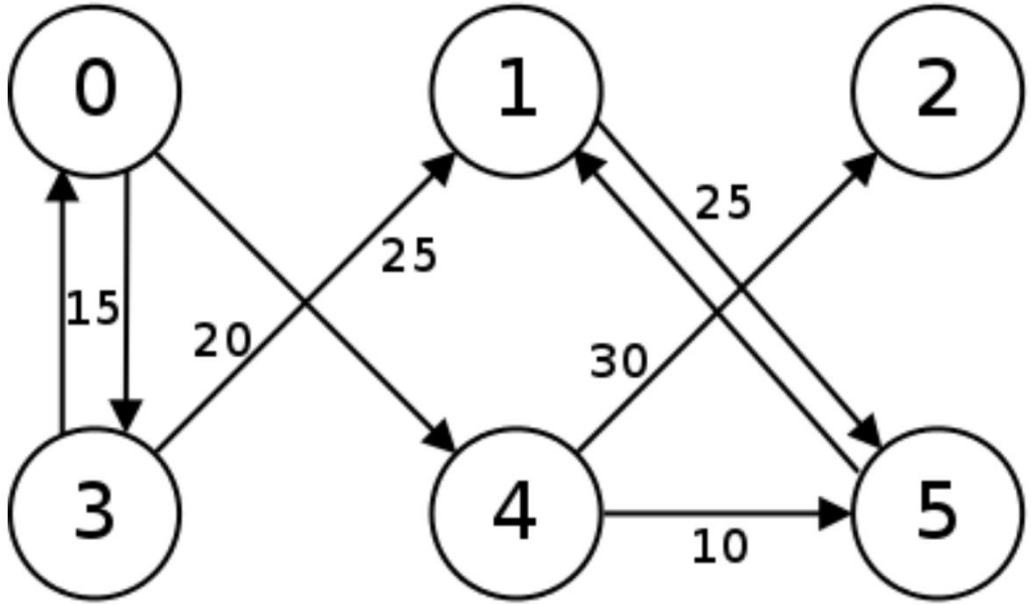
How to represent directed graphs?

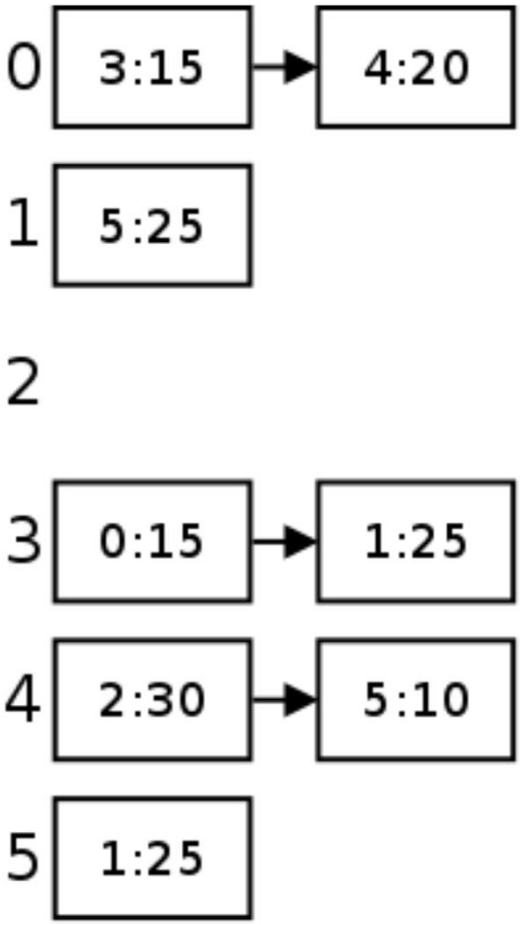
Adjacency list

- For each vertex v, list all its neighbors (vertices to which it is connected by an edge).
  - Array A of $|V|$ linked lists.
  - For $v \in V$, the list $A[v]$ stores neighbors $\{u|(v,u) \in E\}$.
- Directed graph: store only **outgoing** neighbors.
- Undirected graph: stores same edge information in two places



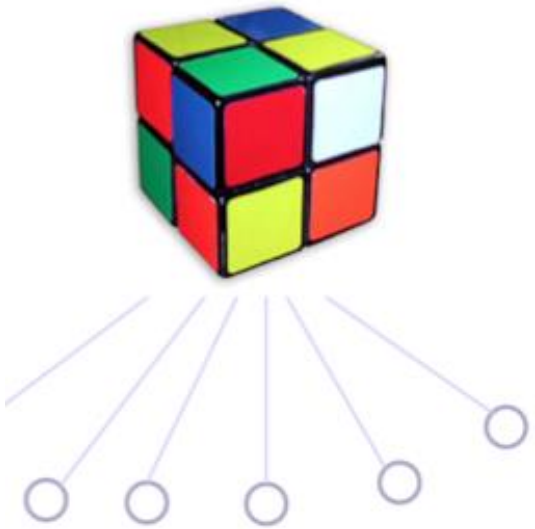
Example of adjacency list with edge weights





Implicit representation

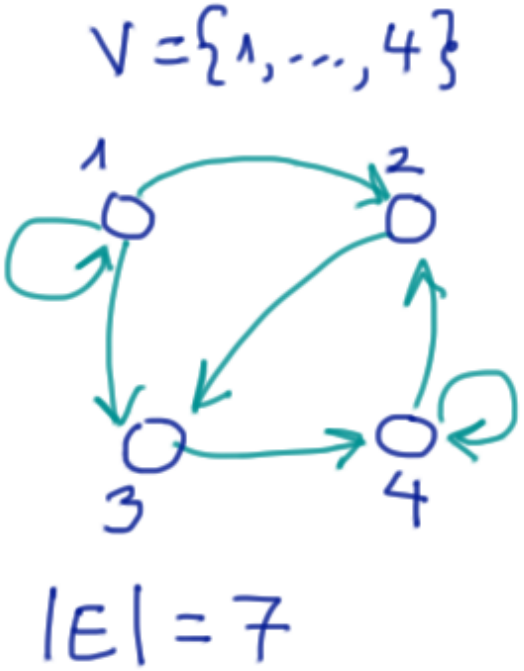
- Don’t store graph at all.
- Implement function $Adj(u)$ that returns list of neighbors or edges of u.
- Requires no space, use it as you need it.
- Can be very efficient.



Trade-off: Space

need $\log n$ bits to represent a vertex

- Assume vertices $\{1, \ldots, n\}$
- Adjacency lists:
  - One list node per edge.
  - So, space is $\Theta((n + m) \log n)$ bits.
- Adjacency matrix:
  - Uses n^2 entries.
  - But each entry can be just one bit.
  - So, space is $\Theta(n^2)$ bits.
- Matrix better only for very dense graphs, i.e., m near n^2.



$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix} \end{array}$$

Trade-off: Time

- Add an edge.
  - Both data structures are $O(1)$.
- Check "is there an edge from u to v"?
  - Matrix is $O(1)$.
  - Adjacency list of u must be scanned.
- Visit all neighbors of u (very common).
  - Adjacency list is $O(\#\text{ neighbors})$.
  - Matrix is $\Theta(n)$.
- Remove edge.
  - Like find + add.